

Metoda Greedy

Prezentare [1]	1
Probleme de planificare	3
A. Planificarea activităților cu minimizarea timpului mediu de așteptare	3
B. Planificarea unui număr maxim posibil de activități – Problema spectacolelor .	5
C. Determinarea numărului minim de resurse necesare pentru a putea desfășura toate activitățile – Problema partiționării intervalelor	8
Problema continuă a rucsacului [1]	9

Prezentare [1]

Metoda Greedy (**greedy**=lacom) este aplicabilă problemelor de optim.

Un tipar posibil de probleme la care se aplică metoda Greedy este următorul:

*Se dă o mulțime finită A . Să se determine o submulțime finită $B \subseteq A$ care satisface o anumită proprietăți **propr**(este **soluție posibilă**) și îndeplinește un criteriu de optim (este **soluție optimă**), adică minimizează/maximizează o **funcție obiectiv f***

Spre exemplu, să considerăm următoarea problemă: Dată o mulțime de intervale, să se determine o submulțime de cardinal maxim de intervale din mulțimea dată care nu se intersectează. Pentru această problemă A = mulțimea de intervale dată, o soluție $S \subseteq A$ este o submulțime de intervale care trebuie să verifice proprietatea că sunt disjuncte două câte două, iar funcția obiectiv care trebuie maximizată este $f(S) = |S|$.

Formalizând, un cadru posibil pentru aplicarea metodei Greedy este următorul: Considerăm mulțimea finită $A = \{a_1, \dots, a_n\}$ și o proprietate **propr** definită pe mulțimea submulțimilor lui A (notată $P(A)$):

$$\text{propr}: P(A) \rightarrow \{\text{True}, \text{False}\} \text{ cu } \begin{cases} \text{propr}(\emptyset) = \text{True} \\ \text{propr}(X) \Rightarrow \text{propr}(Y), \forall Y \subset X \end{cases}$$

O submulțime $S \subset A$ se numește **soluție** (**soluție posibilă**) dacă $\text{propr}(S) = \text{True}$.

Dintre soluțiile posibile dorim să alegem una care optimizează (minimizează/maximizează) o funcție $f: P(A) \rightarrow \mathbb{R}$ dată.

Metoda Greedy urmărește evitarea parcurgerii tuturor submulțimilor (ceea ce ar necesita un timp de calcul exponențial), **mergându-se "direct" spre soluția optimă**. Astfel, soluția se construiește element cu element, elementul ales la un pas pentru a se adăuga în soluție fiind cel care pare cel mai "bun" la acel pas, conform criteriului de optim f . Nu este însă garantată obținerea unei soluții optime, de aceea aplicarea metodei Greedy trebuie însoțită neapărat de o demonstrație.

Distingem două variante generale de aplicare a metodei Greedy: cu prelucrare inițială a elementelor din A (de obicei ordonare) sau fără:

$S \leftarrow \emptyset$ for $i=1, n$ $x \leftarrow \text{alege}(A)$ $A \leftarrow A \setminus \{x\}$ if $\text{propr}(S \cup \{x\}) = \text{True}$ $S \leftarrow S \cup \{x\}$	prelucreaza (A) $S \leftarrow \emptyset$ for x in A if $\text{propr}(S \cup \{x\}) = \text{True}$ $S \leftarrow S \cup \{x\}$ return S
--	---

Observații:

- în algoritm nu apare funcția f
- dificultatea constă în a concepe procedurile `alege`, respectiv `prel`, în care este "ascunsă" funcția f .

Exemplul 1 Se consideră mulțimea de valori reale $A = \{a_1, \dots, a_n\}$. Să se determine o submulțime a lui A a cărei sumă a elementelor este maximă.

Soluție. La fiecare pas trebuie să adăugăm la submulțime un element care să crească suma elementelor din submulțime cu o valoare cât mai mare. Astfel, vom adăuga inițial la submulțime elementul cel mai mare din A . Apoi vom parcurge mulțimea și vom selecta numai elementele pozitive, dacă astfel de elemente există (dacă nu există elemente pozitive, soluția este dată de cel mai mare element din mulțime). Corectitudinea soluției este evidentă.

Exemplul 2 Se consideră mulțimea de valori reale $A = \{a_1, \dots, a_n\}$ și $k < n$. Să se determine o submulțime a lui A de cardinal k a cărei sumă a elementelor este maximă.

Soluție. Vom alege în S cele mai mari k elemente.

Fără o prelucrare inițială a elementelor, pseudocodul este următorul:

```
S ← ∅
for i=1,n
    x ← maxim(A)
    A ← A \ {x} -> eliminam sau marcam x
    S ← S ∪ {x}
    if |S| = k
        stop (break)
```

Complexitatea acestui algoritm este $O(nk)$ (alegem de k ori maximul), care oscilează între $O(n)$ și $O(n^2)$ după cum valoarea lui k este mai apropiată de 1 sau de n .

Am putea însă ordona de la început elementele din A descrescător, pentru a alege mai simplu pe cele mai mari k (fiind primele k din ordonare), și în acest caz complexitatea va fi $O(n \log_2(n) + k) = O(n \log_2(n))$ (deoarece $k \leq n \leq n \log_2(n)$):

```
sorteaza_descrescator(A)
S ← ∅
for x in A
    S ← S ∪ {x}
    if |S| = k
        stop (break)
return S
```

Exemplul 3 (Contraexemplu) Se consideră mulțimea $A = \{a_1, \dots, a_n\}$ cu elemente pozitive. Să se determine o submulțime a lui A de sumă maximă, dar cel mult egală cu o valoare M dată.

Dacă alegem la fiecare pas cel mai mare element care poate fi adăugat la soluție fără a depăși M , pentru $A = \{6, 3, 4, 2\}$ și $M = 7$ obținem $\{6\}$. Dar soluția optimă este $\{3, 4\}$ cu suma egală cu 7.

Continuăm cu prezentarea unor exemple clasice.

Probleme de planificare

Vom prezenta câteva tipuri de probleme de planificare care se pot rezolva cu ajutorul metodei Greedy. Cadrul comun este următorul.

Avem n activități care se desfășoară utilizând o resursă comună (procesor, sală de spectacole, bucătar/evaluator de proiecte care execută/evaluează comenzi/proiecte pe rând etc). La un moment dat **doar o activitate poate folosi resursa**, iar o activitate începută trebuie finalizată (nu poate fi întreruptă)

A. **Planificarea activităților astfel încât să fie minimizat timpul mediu de așteptare** (timpul de așteptare necesar finalizării unei activități este dat de suma dintre timpii de execuție al activităților programate înaintea ei și timpul de execuție al ei, deoarece o activitate trebuie să aștepte se finalizeze activitățile programate înainte pentru a putea începe), dacă se cunosc $t_1, \dots, t_n$ duratele de desfășurare a activităților

B. **Planificarea unui număr maxim posibil de activități**, dacă se cunosc intervalele în care trebuie să se desfășoare aceste activități – **Problema spectacolelor**

C. **Determinarea numărului minim de resurse necesare pentru a putea desfășura toate activitățile**, dacă se cunosc intervalele în care trebuie să se desfășoare aceste activități – **Problema partiționării intervalelor**

A. Planificarea activităților cu minimizarea timpului mediu de așteptare

Avem n activități (comenzi date simultan de n clienți la un restaurant) care se desfășoară utilizând o resursă comună (un bucătar care trebuie să execute comenzile, pe rând, câte una sau un evaluator de proiecte care evaluează pe rând proiecte depuse, un procesor etc)

Știind durata fiecărei activități $t_1, t_2, \dots, t_n$ (cât îi ia bucătarului să facă fiecare comandă sau evaluatorului să evalueze fiecare proiect), să se determine ordinea în care trebuie executate activitățile astfel încât timpul mediu de așteptare să fie minim: o activitate trebuie să aștepte se finalizeze activitățile programate înainte pentru a putea începe (un client trebuie să aștepte ca bucătarul să termine toate comenzile pe care le face înainte plus comanda lui pentru a fi servit; o persoană care a depus un proiect primește răspunsul evaluării proiectului său după ce au fost evaluate toate proiectele programate înainte plus proiectul său).

Pentru activitatea i notăm cu a_i timpul de așteptare până la finalizarea activității i (în raport cu ordinea aleasă a activităților).

De exemplu, să presupunem că avem $n = 4$ activități cu duratele $t_1 = 5, t_2 = 1, t_3 = 7, t_4 = 2$. Timpul mediu de așteptare depinde de ordinea în care se execută activitățile. Pentru exemplificarea modului în care se calculează acest timp vom considera câteva ordonări și vom calcula timpul mediu pentru fiecare dintre aceste ordonări.

- dacă activitățile se execută în ordinea 1, 2, 3, 4, timpul de așteptare pentru fiecare activitate este calculat în următorul tabel:

activitatea	durata	Timpul de așteptare
1	$t_1 = 5$	$a_1 = t_1 = 5$
2	$t_2 = 1$	$a_2 = a_1 + t_2 = 5 + 1 = 6$ $= t_1 + t_2$
3	$t_3 = 7$	$a_3 = a_2 + t_3 = 6 + 7 = 13$ $= (t_1 + t_2) + t_3$
4	$t_4 = 2$	$a_4 = a_3 + t_4 = 13 + 2 = 15$ $= (t_1 + t_2 + t_3) + t_4$

Timpul mediu de așteptare pentru ordonarea 1, 2, 3, 4 va fi

$$\frac{a_1 + a_2 + a_3 + a_4}{4} = \frac{5 + 6 + 13 + 15}{4} = \frac{39}{4}$$

- dacă activitățile se execută în ordinea 2, 4, 1, 3, timpul de așteptare pentru fiecare activitate este calculat în următorul tabel:

activitatea	durata	Timpul de așteptare
2	$t_2 = 1$	$a_2 = t_2 = 1$
4	$t_4 = 2$	$a_4 = a_2 + t_4 =$ $= t_2 + t_4 = 1 + 2 = 3$
1	$t_1 = 5$	$a_1 = a_4 + t_1 = 3 + 5 = 8$ $= (t_2 + t_4) + t_1$
3	$t_3 = 7$	$a_3 = a_1 + t_3 =$ $= (t_2 + t_4 + t_1) + t_3 = 8 + 7 = 15$

Timpul mediu de așteptare pentru ordonarea 1, 2, 3, 4 va fi

$$\frac{a_2 + a_4 + a_1 + a_3}{4} = \frac{1 + 3 + 8 + 15}{4} = \frac{27}{4}$$

Astfel, a determina o soluție este echivalent cu a determina o permutare $\sigma \in S_n$ pentru care se minimizează timpul mediu de așteptare:

$$\begin{aligned} T(\sigma) &= \frac{1}{n} (a_{\sigma(1)} + \dots + a_{\sigma(n)}) = \\ &= \frac{1}{n} [t_{\sigma(1)} + (t_{\sigma(1)} + t_{\sigma(2)}) + \dots + (t_{\sigma(1)} + \dots + t_{\sigma(n)})] = \\ &= \frac{1}{n} \sum_{k=1}^n (n - k + 1) t_{\sigma(k)} \end{aligned}$$

Soluție Greedy: După ce criteriu alegem prima activitate, sau, mai general, activitatea planificată la pasul i ? Deoarece după prima activitate așteaptă toate celelalte, vom alege ca prima activitate efectuată să fie cea cu durata cea mai mică. Raționăm similar la fiecare pas și alegem mereu activitatea cu durata cea mai mică dintre cele rămase. Astfel, strategia Greedy pentru această problemă este de a planifica activitățile în ordine crescătoare după durată.

Corectitudine [1]: Vom demonstra că soluția dată de strategia Greedy este optimă. Pentru aceasta să notăm activitățile astfel încât

$$t_1 \leq t_2 \leq \dots \leq t_n.$$

Atunci soluția Greedy este permutarea identică id .

Fie $\sigma \in S_n$ o așezare optimă (σ minimizează funcția T).

Presupunem prin absurd că $T(\sigma) < T(id)$, adică soluția Greedy nu este optimă.

Să observăm întâi că dacă σ ar diferi de id doar prin ordinea în care alege activitățile în caz de egalitate, atunci am avea $T(\sigma) = T(id)$.

Evidențiem o diferență între σ și id , adică o inversiune în σ și arătăm că eliminând această diferență (permutând elementele inversiunii) obținem o soluție mai bună decât σ .

Atunci $\exists i < j$ cu $t_{\sigma(i)} > t_{\sigma(j)}$. Considerăm permutarea σ' obținută din σ prin interschimbarea elementelor de pe pozițiile i și j (este suficient să considerăm $j = i + 1$; permutarea σ' are mai puține inversiuni, deci este "mai apropiată" de soluția Greedy id decât σ).

Arătăm că $T(\sigma') < T(\sigma)$, ceea ce contrazice optimalitatea lui σ (soluția σ' "mai apropiată" de soluția Greedy are timpul asociat mai mic decât σ).

Folosind formulele pentru $T(\sigma)$ și $T(\sigma')$ și reducând termenii în diferența $T(\sigma) - T(\sigma')$, obținem:

$$\begin{aligned} T(\sigma) - T(\sigma') &= \frac{1}{n} \sum_{k=1}^n (n-k+1) t_{\sigma(k)} - \frac{1}{n} \sum_{k=1}^n (n-k+1) t_{\sigma'(k)} = \\ &= \frac{1}{n} \left[(n-i+1) t_{\sigma(i)} + (n-j+1) t_{\sigma(j)} - (n-i+1) t_{\sigma(j)} - (n-j+1) t_{\sigma(i)} \right] = \\ &= \frac{1}{n} \left[(j-i) t_{\sigma(i)} + (i-j) t_{\sigma(j)} \right] = \frac{(j-i)(t_{\sigma(i)} - t_{\sigma(j)})}{n} > 0 \end{aligned}$$

(deoarece $j > i$ și $t_{\sigma(i)} > t_{\sigma(j)}$)

Rezultă că $T(\sigma') < T(\sigma)$, contradicție.

Variante. Generalizări (v. laborator+seminar)

1. Fiecare text are asociată o frecvență de citire
2. Avem la dispoziție p benzi ($p \geq 1$)

B. Planificarea unui număr maxim posibil de activități – Problema spectacolelor [2]

• Problema selectării activităților (problema spectacolelor) [2]

Să presupunem că avem o mulțime $A = \{1, 2, \dots, n\}$ de n activități (spectacole) care doresc să folosească aceeași resursă (sala de spectacole). Această resursă poate fi folosită de o singură activitate la un moment dat. Fiecare activitate i are un timp de start s_i și un timp de terminare t_i , deci se poate desfășura doar în intervalul $[s_i, t_i)$. Problema selectării activităților (spectacolelor) constă în selectarea unei mulțimi de activități compatibile între ele de cardinal maxim (compatibile = având intervalele de desfășurare disjuncte).

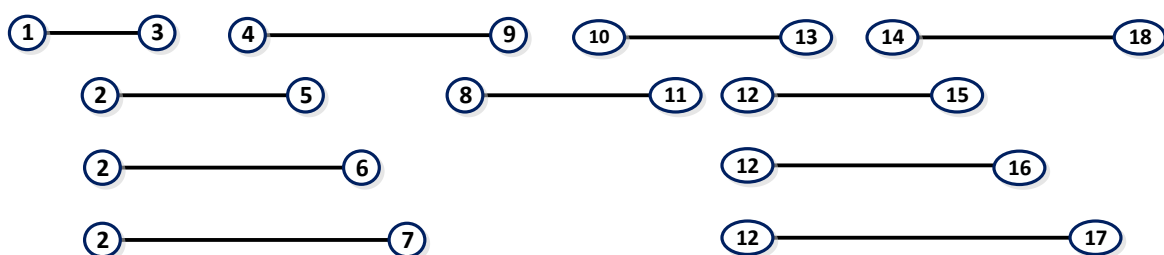
Variantă de enunț: Știind intervalele de desfășurare a unor conferințe la care dorim să asistăm în timpul unei zile (sau emisiuni pe care dorim să le vizionăm), să se determine numărul maxim de conferințe la care putem participa (= cu intervale de desfășurare disjuncte).

VARIANTĂ DE ENUNȚ CU INTERVALE: Se consideră o mulțime A de intervale închise. Să se determine o submulțime de cardinal maxim de intervale disjuncte din A

Soluție. După ce criteriu alegem prima activitate, sau, mai general, activitatea planificată la pasul curent?

Posibile criterii „greedy” de alegere:

- activitatea cu durata cea mai mică, pentru a ocupa cât mai puțin resursa – nu este corect, deoarece acest criteriu nu ține cont de intervalul în care se desfășoară activitatea, care se poate intersecta cu intervalele altor activități. Spre exemplu, pentru activitățile cu intervalele $[1,10)$, $[9,12)$, $[13,18)$ soluția optimă este formată cu activitățile 1 și 3 (nu conține activitatea 2 cu lungimea cea mai mică)
- activitatea care începe cel mai devreme, pentru a ocupa cât mai repede resursa – nu este corect, această activitate poate avea durată mare: contraexemplu: $[1,10)$, $[2,4)$, $[5,8)$
- activitatea care este compatibilă cu cât mai multe activități (sau, echivalent, se intersectează cu cât mai puține intervalele de desfășurare ale celorlalte activități), pentru ca prin programarea ei să fie excluse din lista activităților care se mai pot programa cât mai puține activități – nici acest criteriu nu este corect, pentru că o mare parte dintre activitățile cu care este compatibilă pot să nu fie compatibile două câte două, ca în exemplul din desenul de mai jos (unde sunt indicate intervalele de desfășurare ale activităților). Intervalul $[8,11)$ se intersectează doar cu două intervale, dar soluția optimă este formată cu activitățile cu intervalele $[1,3)$, $[4,9)$, $[10,13)$, $[14,18)$



Strategia Greedy (corectă) pentru rezolvarea acestei probleme este următoarea: la fiecare pas este selectată o activitate nouă care se termină cel mai **devreme (cu timpul de terminare cel mai mic)** compatibilă cu activitățile deja selectate, pentru a lăsa după terminarea ei **resursa liberă cât mai mult timp**. Pentru aceasta este util să procesăm inițial datele, mai exact să ordonăm activitățile crescător după timpul de terminare.

Pentru a simplifica scrierea pseudocodului și justificarea corectitudinii, să presupunem că activitățile sunt ordonate crescător după timpul de terminare: $t_1 \leq t_2 \leq \dots \leq t_n$

Algoritmul Greedy este descris de următoarea funcție, scrisă în pseudocod.

```
function selectie_activitati(s, t)
    S ← {1}
    us ← 1 //indicele ultimei activitati selectate
    pentru ac = 2, n executa //activitatea curenta
        daca sac ≥ tus atunci // tus = max{ tk , k ∈ S}
            S ← S ∪ {ac}
            us ← ac
    return S
```

În mulțimea S se introduc activitățile care au fost selectate. Variabila us indică ultima activitate introdusă în S .

Deoarece activitățile sunt considerate în ordinea crescătoare a timpilor lor de terminare, t_{us} va reprezenta timpul maxim de terminare a oricărei activități din S :

$$t_{us} = \max\{t_k, k \in S\}$$

De aceea, pentru a vedea dacă activitatea curentă a_c este compatibilă cu toate celelalte activități existente la momentul curent în S este suficient ca $s_{a_c} \geq t_{us}$.

Corectitudine. Presupunem (ca și în algoritm) că activitățile sunt ordonate crescător după timpul de terminare: $t_1 \leq t_2 \leq \dots \leq t_n$. Fie $S = \{g_1=1, \dots, g_m\}$ (cu $g_1 < \dots < g_m$) soluția Greedy.

Varianta 1 [2]:

Vom folosi o strategie întâlnită în multe demonstrații de corectitudine pentru algoritmi de tip Greedy, similară celei din problema anterioară. Pentru a demonstra că soluția Greedy S este optimă considerăm o soluție optimă O care are un număr maxim de elemente inițiale în comun cu Greedy (o soluție optimă care “seamănă” cel mai mult cu soluția Greedy). Evidențiem prima diferență dintre O și S și arătăm că eliminând această diferență (înlocuind elementul din O care nu este în soluția Greedy cu unul din S – exchange argument) obținem o soluție cel puțin la fel de bună ca O , care are însă mai multe elemente în comun cu O (deci **O se poate transforma element cu element printr-o strategie de tip “exchange argument” în S**).

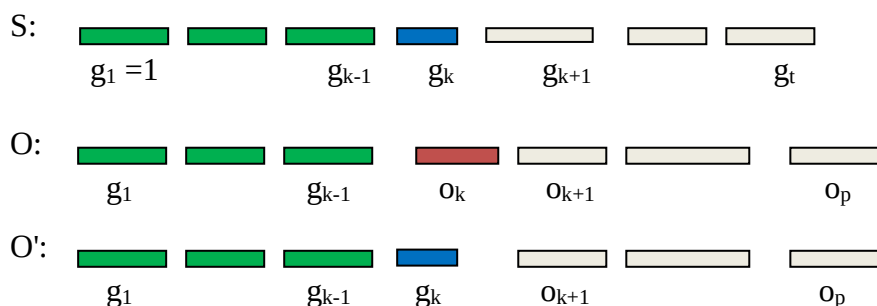
Considerăm o soluție optimă $O = \{o_1, \dots, o_p\}$ (cu activitățile notate astfel încât $o_1 < \dots < o_p$, deci $t_{o_1} \leq \dots \leq t_{o_p}$) care are **un număr maxim de elemente (inițiale) în comun cu soluția Greedy S** . Avem $m \leq p$. Presupunem prin absurd $|O| > |S|$.

Atunci există un indice $k \leq m$ astfel încât $g_k \neq o_k$ (nu putem avea $S \subset O$ și $m < p$ deoarece activitatea o_{m+1} este compatibilă cu toate activitățile din $\{o_1, \dots, o_m\} = \{g_1, \dots, g_m\}$ și algoritmul Greedy `selectie_activitati` ar mai fi avut activități compatibile cu $\{g_1, \dots, g_m\}$ din care să selecteze, deci ar fi furnizat o soluție cu mai multe elemente).

Fie $k \leq m$ cel mai mic indice astfel încât $g_k \neq o_k$ (prima poziție pe care soluția greedy S și soluția optimă O diferă). La pasul la care `selectie_activitati` a ales activitatea g_k , activitatea o_k era de asemenea neselectată și compatibilă cu activitățile deja selectate $g_1 = o_1, \dots, g_{k-1} = o_{k-1}$. Deoarece activitatea g_k a fost cea selectată, rezultă că $t_{g_k} \leq t_{o_k}$ (g_k a fost selectată deoarece avea timpul de terminare mai mic), deci g_k este compatibilă cu $o_{k+1}, \dots, o_m$ (deoarece acestea încep după ce se termină o_k) – v. desenul de mai jos:

$$t_{g_k} \leq t_{o_k} \leq s_{o_{k+1}} \leq t_{o_{k+1}} \leq \dots \leq s_{o_p} \leq t_{o_p}$$

Atunci putem înlocui în soluția optimă O activitatea o_k cu g_k și obținem tot o soluție posibilă $O' = O - \{o_k\} \cup \{g_k\}$ (formată din activități compatibile din A). Avem $|O'| = |O|$, deci O' este tot o soluție optimă, dar care are mai multe elemente în comun cu soluția greedy S , contradicție.



Varianta 2 (similar) – Demonstrăm prin inducție după n că algoritmul greedy

`selectie_activitati` construiește o soluție optimă.

Pentru $n = 0, 1$ afirmația este evidentă.

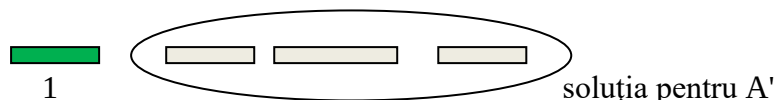
Fie $n \geq 2$. Presupunem că algoritmul `selectie_activitati` construiește o soluție optimă pentru orice mulțime de activități A' cu $|A'| < n$

Fie A o mulțime de n activități.

a) **Există o soluție optimă care conține prima activitate selectată de algoritmul greedy, anume $g_1 = 1$**

Într-adevăr, fie $O = \{o_1, \dots, o_p\}$ o soluție optimă (o submulțime de cardinal maxim de activități compatibile) cu activitățile notate astfel încât $o_1 < \dots < o_p$, deci $t_{o_1} \leq \dots \leq t_{o_p}$. Dacă activitatea 1 aparține lui O , atunci afirmația a) este adevărată. Altfel considerăm $O' = O - \{o_1\} \cup \{1\}$. Deoarece $t_1 \leq t_{o_1}$, activitatea 1 este compatibilă cu toate celelalte activități din $O - \{o_1\}$, deci O' este o submulțime de activități compatibile cu $|O'| = |O|$, adică o soluție optimă care conține activitatea 1.

b) Fie $A' = \{k \in S, s_k \geq t_1\}$ mulțimea activităților care încep după ce se termină activitatea 1 (compatibile cu activitatea 1). Conform ipotezei de inducție soluția S' construită de algoritmul `selectie_activitati` pentru A' este soluție optimă. Rezultă că $S = S' \cup \{1\}$ este soluție optimă pentru A (altfel, conform punctului a) ar exista o soluție optimă O pentru A care conține activitatea 1 cu cardinal mai mare decât S : $|O| > |S|$; dar atunci $O - \{1\}$ este soluție posibilă pentru A' cu $|O - \{1\}| > |S - \{1\}| = |S'|$, ceea ce contrazice optimalitatea lui S')



Variante. Generalizări

1. Problema partiționării intervalelor (- programarea tuturor activităților folosind un numărul minim de resurse = partiționarea unei mulțimi de intervale într-un număr minim de submulțimi de intervale disjuncte două câte două -v. laborator, seminar, [2])
2. Cazul în care fiecare interval are asociată o pondere (un profit asociat activității) și se cere determinarea unei submulțimi de intervale disjuncte de pondere maximă – la programare dinamică

C. Determinarea numărului minim de resurse necesare pentru a putea desfășura toate activitățile – Problema partiționării intervalelor [2]

Să presupunem că avem n activități (spectacole) care pentru a se desfășura au nevoie de o resursă (sală de spectacole). Această resursă poate fi folosită de o singură activitate la un moment dat. Fiecare activitate i are un timp de start s_i și un timp de terminare t_i , deci se poate desfășura doar în intervalul $[s_i, t_i)$. Astfel, pe o resursă se pot planifica doar activități cu intervalele de desfășurare disjuncte. Să se determine numărul minim k de resurse (săli de spectacole) de care este nevoie pentru a efectua toate activitățile (spectacolele) și o planificare a acestor activități pe cele k resurse.

Exemplu: pentru $n=3$ spectacole, care trebuie să se desfășoare în intervalele: $[10, 14)$, $[12, 16)$, respectiv $[17, 18)$, sunt necesare 2 săli, o programare optimă fiind:

- o Sala 1: $[10, 14)$ – spectacolul 1, $[17, 18)$ – spectacolul 3
- o Sala 2: $[12, 16)$ – spectacolul 2

Variantă de enunț cu intervale – Problema partiționării intervalelor: Se consideră o mulțime A de intervale închise. Să se împartă (partiționeze) această mulțime de intervale într-un **număr minim de submulțimi** cu proprietatea că oricare două intervale dintr-o submulțime nu se intersectează și să se afișeze aceste submulțimi

Soluție – Temă. v. seminar+laborator

Problema continuă a rucsacului [1]

Se consideră un rucsac de capacitate (greutate) maximă G și n obiecte caracterizate prin:

- greutatea lor $g_1, \dots, g_n$;
- câștigurile $c_1, \dots, c_n$ obținute la încărcarea lor în totalitate în rucsac.

Din fiecare obiect poate fi încărcată orice fracțiune a sa (și valoarea asociată va fi acea fracțiune din câștigul total). Să se determine o modalitate de încărcare de (fracțiuni de) obiecte în rucsac, astfel încât câștigul total să fie maxim.

De exemplu, pentru $G = 8$, $n = 4$, obiectul 1 cu greutatea 4 și câștigul 8, obiectul 2 cu greutatea 8 și câștigul 12, obiectul 3 cu greutatea 3 și câștigul 6 și obiectul 4 cu greutatea 10 și câștigul 10 o soluție optimă este dată de vectorul de fracțiuni $(1, 1/8, 1, 0)$, adică se vor încărca obiectele 1 și 3 întregi și $1/8$ din obiectul 2, iar câștigul total va fi $8 + 1/8 \cdot 12 + 6 = 15,5$

Variantă de enunț: T =timp total de funcționare a unei resurse, n activități cu durata d_i și profitul p_i . O activitate începută poate fi întreruptă obținându-se un profit parțial.

Soluție [1]. Prin *soluție* înțelegem un vector $x = (x_1, \dots, x_n)$ cu

$$\begin{cases} x_i \in [0, 1], \forall i \\ \sum_{i=1}^n g_i x_i \leq G \end{cases}$$

O *soluție optimă* este soluție care maximizează funcția $f(x) = \sum_{i=1}^n c_i x_i$.

Dacă suma greutateilor obiectelor este mai mică decât G , atunci încărcăm toate obiectele: $x = (1, \dots, 1)$. De aceea **presupunem în continuare că $g_1 + \dots + g_n > G$** .

Exemplu

Cum alegem obiectul care trebuie încărcat la pasul curent? Alegem dintre obiectele rămase pe cel mai valoros **pe unitatea de greutate** (cu valoarea cea mai mare a raportului c/g) și încărcăm cât putem de mult din el.

Astfel, pentru a implementa strategia Greedy, ar fi util să ordonăm inițial obiectele descrescător după câștigul la unitatea de greutate. Algoritmul constă în încărcarea în această ordine a obiectelor, atâta timp cât nu se depășește greutatea G , ultimul obiect adăugat putând fi eventual încărcat parțial. Mai exact, dacă presupunem obiectele ordonate astfel încât

$$\frac{c_1}{g_1} \geq \frac{c_2}{g_2} \geq \dots \geq \frac{c_n}{g_n}$$

algoritmul este următorul:

$G1 \leftarrow G$ { **$G1$ reprezintă greutatea disponibilă** }

for $i=1, n$

if $g_i \leq G1$

$x_i \leftarrow 1$; $G1 \leftarrow G1 - g_i$

else

$x_i \leftarrow G1 / g_i$;

```

    for j=i+1, n
        xj ← 0
    stop
write(x)

```

Am obținut deci soluția greedy $x = (1, \dots, 1, x_j, 0, \dots, 0)$ cu $x_j \in [0, 1)$.

Corectitudine [1]. –Arătăm că soluția astfel obținută este optimă.

Fie y soluția optimă **care are o subsecvență inițială maximă în comun cu x :**

$$y = (\dots, y_k, \dots) \text{ cu } \begin{cases} \sum_{i=1}^n g_i y_i = G \\ \sum_{i=1}^n c_i y_i \text{ maxim} \end{cases}$$

Dacă $y \neq x$, fie k **prima poziție** pe care $y_k \neq x_k$. Atunci avem

- $k \leq j$: pentru $k > j$ se depășește G .
- $y_k < x_k$

Varianta 1 de demonstrație.

Deoarece $y_k < x_k$, există un indice $t > k$ cu $y_t > x_t$ și $c_k/g_k > c_t/g_t$ (dacă x și y ar diferi doar prin fracțiuni alese din obiecte egale, ambele ar fi optime). Considerăm o soluție y' în care luăm o fracțiune mai mare $y'_k = y_k + \varepsilon$ din obiectul k și una mai mică din obiectul t astfel încât greutatea totală corespunzătoare lui y' să fie tot G , deci $y'_t = y_t - \varepsilon g_k/g_t$.

Avem $f(y') - f(y) = \varepsilon c_k - (\varepsilon g_k/g_t) c_t = \varepsilon g_k (c_k/g_k - c_t/g_t) > 0$, contradicție.

Varianta 2 de demonstrație

Considerăm o soluție y' cu **mai multe elemente inițiale în comun** cu soluția Greedy x , obținută din y înlocuind y_k cu x_k și păstrând o fracțiune mai mică din obiectele $k+1, \dots, n$, astfel încât să obținem tot greutatea G : $y' = (y_1, \dots, y_{k-1}, x_k, \alpha y_{k+1}, \dots, \alpha y_n)$ cu $\alpha < 1$, astfel încât:

$$g_k (x_k - y_k) = (1 - \alpha) (g_{k+1} y_{k+1} + \dots + g_n y_n) \quad (*)$$

Comparăm performanța lui y' cu cea a lui y pentru a arăta că soluția y' este cel puțin la fel de bună ca y , dar cu mai multe elemente în comun cu soluția greedy x , ceea ce contrazice alegerea lui y . Avem

$$\begin{aligned} f(y') - f(y) &= c_k x_k + \alpha c_{k+1} y_{k+1} + \dots + \alpha c_n y_n - (c_k y_k + c_{k+1} y_{k+1} + \dots + c_n y_n) = \\ &= c_k (x_k - y_k) - (1 - \alpha) (c_{k+1} y_{k+1} + \dots + c_n y_n) = \\ &= c_k / g_k \cdot g_k (x_k - y_k) + (\alpha - 1) (c_{k+1} / g_{k+1} \cdot g_{k+1} y_{k+1} + \dots + c_n / g_n \cdot g_n y_n) = \end{aligned}$$

Dar $1 - \alpha > 0$ și $c_k/g_k \geq c_s/g_s, \forall s > k$. Atunci

$$f(y') - f(y) \geq c_k / g_k [g_k (x_k - y_k) + (\alpha - 1) (c_{k+1} y_{k+1} + \dots + c_n y_n)] = 0 \text{ (din (*))},$$

deci $f(y') \geq f(y)$. Rezultă că și y' este soluție optimă ($f(y)$ este maxim), contradicție cu alegerea lui y , deoarece y' are mai multe elemente inițiale în comun cu x decât y .

Variante. Generalizări

Problema discretă a rucsacului diferă de cea continuă prin faptul că fiecare obiect poate fi încărcat numai în întregime în rucsac.

Să observăm că aplicarea metodei Greedy eșuează în acest caz. Într-adevăr, aplicarea ei pentru: $G=5$, $n=3$ și $g=(4, 3, 2)$, $c=(6, 4, 2.5)$

are ca rezultat încărcarea primului obiect; câștigul obținut este 6. Dar încărcarea ultimelor două obiecte conduce la un câștig mai mare 6.5.

Bibliografie

1. Horia Georgescu. **Tehnici de programare**. Editura Universității din București 2005
2. **Jon Kleinberg, Éva Tardos** - Algorithm Design, 2005 Addison-Wesley Professional
 Slideu-rile oficiale pentru această carte se pot găsi la
<http://www.cs.princeton.edu/~wayne/kleinberg-tardos/pearson/>
 o versiune îmbogățită a acestora este accesibilă la adresa
<http://www.cs.princeton.edu/~wayne/kleinberg-tardos>